

Lab Assignment 11.1

Name: K.SAI VARDHAN REDDY

Roll No: 2503A51266

Task 1

Question: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Prompt used for AI: Generate a Python Stack class using a list. Include push, pop, peek, and is_empty methods. Add clear docstrings, handle pop/peek on an empty stack safely, and provide a short example that demonstrates all four operations.

Answer / Code:

```
class Stack:
    """A simple LIFO stack implemented using a Python list."""

    def __init__(self):
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the top item, or return None if empty."""
        if self.is_empty():
            return None
        return self.items.pop()

    def peek(self):
        """Return the top item without removing it, or None if empty."""
        if self.is_empty():
            return None
        return self.items[-1]

    def is_empty(self):
        """Return True when the stack contains no items."""
        return len(self.items) == 0

stack = Stack()
stack.push(10)
stack.push(20)
print("Top:", stack.peek())
print("Popped:", stack.pop())
print("Empty:", stack.is_empty())
```

Explanation: The stack follows LIFO (Last In, First Out). The list's append/pop operations are used for the top of the stack.

Task 2

Question: Use AI to implement a Queue using Python lists.

Prompt used for AI: Generate a Python Queue class using a list. Include enqueue, dequeue, peek, and size methods. Use FIFO behavior, handle an empty queue safely, add docstrings, and include a small demonstration.

Answer / Code:

```
class Queue:
    """A simple FIFO queue implemented using a Python list."""

    def __init__(self):
        self.items = []

    def enqueue(self, item):
        """Add an item to the rear of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the front item, or return None if empty."""
        if not self.items:
            return None
        return self.items.pop(0)

    def peek(self):
        """Return the front item without removing it, or None if empty."""
        if not self.items:
            return None
        return self.items[0]

    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)

queue = Queue()
queue.enqueue("Student 1")
queue.enqueue("Student 2")
print("Front:", queue.peek())
print("Removed:", queue.dequeue())
print("Size:", queue.size())
```

Explanation: The queue follows FIFO (First In, First Out): the first inserted item is the first item removed.

Task 3

Question: Use AI to generate a Singly Linked List with insert and display methods.

Prompt used for AI: Generate a Python singly linked list with a Node class and LinkedList class. Implement insert to add nodes at the end and display to print the list. Add docstrings and demonstrate the result.

Answer / Code:

```
class Node:
    """A node containing data and a reference to the next node."""

    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    """A singly linked list."""

    def __init__(self):
        self.head = None

    def insert(self, data):
        """Insert a new node at the end of the list."""
```

```

        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

    def display(self):
        """Display all values in the linked list."""
        current = self.head
        values = []

        while current:
            values.append(str(current.data))
            current = current.next

        print(" -> ".join(values))

```

```

linked_list = LinkedList()
linked_list.insert(10)
linked_list.insert(20)
linked_list.insert(30)
linked_list.display()

```

Explanation: Each node stores data and a reference to the next node. The display method traverses the list from head to the end.

Task 4

Question: Use AI to create a BST with insert and in-order traversal methods.

Prompt used for AI: Generate a Python Binary Search Tree with a Node class and BST class. Implement recursive insert and in-order traversal. Add docstrings and demonstrate the sorted traversal output.

Answer / Code:

```

class BSTNode:
    """A node in a binary search tree."""

    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BST:
    """Binary Search Tree supporting recursive insertion and in-order traversal."""

    def __init__(self):
        self.root = None

    def insert(self, key):
        """Insert a key into the BST."""
        self.root = self._insert_recursive(self.root, key)

    def _insert_recursive(self, node, key):
        if node is None:
            return BSTNode(key)

```

```

        if key < node.key:
            node.left = self._insert_recursive(node.left, key)
        elif key > node.key:
            node.right = self._insert_recursive(node.right, key)

        return node

    def inorder(self):
        """Return the keys using in-order traversal."""
        result = []

        def traverse(node):
            if node:
                traverse(node.left)
                result.append(node.key)
                traverse(node.right)

        traverse(self.root)
        return result

tree = BST()
for value in [50, 30, 70, 20, 40, 60, 80]:
    tree.insert(value)

```

print("In-order:", tree.inorder())

Explanation: In-order traversal of a BST visits values in ascending order when the tree contains unique numeric keys.

Task 5

Question: Use AI to implement a hash table with basic insert, search, and delete methods.

Prompt used for AI: Generate a Python hash table with insert, search, and delete methods. Use chaining to handle collisions.

Implement a simple hash function, add docstrings, and demonstrate insertion, searching, and deletion.

Answer / Code:

```

class HashTable:
    """A simple hash table using chaining for collision handling."""

    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert or update a key-value pair."""
        index = self._hash(key)

        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                return

        self.table[index].append([key, value])

    def search(self, key):
        """Return the value for a key, or None if it is not present."""

```

```

        index = self._hash(key)

        for pair in self.table[index]:
            if pair[0] == key:
                return pair[1]

        return None

    def delete(self, key):
        """Delete a key and return True if it existed."""
        index = self._hash(key)

        for pair in self.table[index]:
            if pair[0] == key:
                self.table[index].remove(pair)
                return True

        return False

table = HashTable()
table.insert("CS101", "Python")
table.insert("CS102", "Data Structures")
print(table.search("CS101"))
print(table.delete("CS101"))
print(table.search("CS101"))

```

Explanation: Chaining stores multiple key-value pairs in the same bucket when their hash indexes collide.

Task 6

Question: Use AI to implement a graph using an adjacency list.

Prompt used for AI: Generate a Python Graph class using an adjacency list. Include methods to add vertices, add edges, and display connections. Use an undirected graph and provide a small demonstration.

Answer / Code:

```

class Graph:
    """Undirected graph represented with an adjacency list."""

    def __init__(self):
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        """Add a vertex if it does not already exist."""
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []

    def add_edge(self, vertex1, vertex2):
        """Add an undirected edge between two vertices."""
        self.add_vertex(vertex1)
        self.add_vertex(vertex2)

        if vertex2 not in self.adjacency_list[vertex1]:
            self.adjacency_list[vertex1].append(vertex2)

        if vertex1 not in self.adjacency_list[vertex2]:
            self.adjacency_list[vertex2].append(vertex1)

    def display(self):
        """Display each vertex and its connected vertices."""
        for vertex, neighbors in self.adjacency_list.items():

```

```
print(f"{vertex}: {' '.join(map(str, neighbors))}")
```

```
graph = Graph()
graph.add_edge("A", "B")
graph.add_edge("A", "C")
graph.add_edge("B", "D")
graph.display()
```

Explanation: An adjacency list stores each vertex with a list of its connected vertices, making it suitable for sparse graphs.

Task 7

Question: Use AI to implement a priority queue using Python's heapq module.

Prompt used for AI: Generate a Python priority queue using heapq. Implement enqueue with a priority, dequeue, and display.

Define clearly how priority is interpreted, add docstrings, and demonstrate the operations.

Answer / Code:

```
import heapq

class PriorityQueue:
    """Priority queue where a smaller priority number is served first."""

    def __init__(self):
        self.items = []

    def enqueue(self, item, priority):
        """Add an item with its priority."""
        heapq.heappush(self.items, (priority, item))

    def dequeue(self):
        """Remove and return the highest-priority item, or None if empty."""
        if not self.items:
            return None
        return heapq.heappop(self.items)

    def display(self):
        """Display queue items in priority order."""
        print(sorted(self.items))

queue = PriorityQueue()
queue.enqueue("Emergency", 1)
queue.enqueue("Normal", 3)
queue.enqueue("Important", 2)

queue.display()
print("Served:", queue.dequeue())
```

Explanation: The example treats a smaller number as a higher priority. heapq keeps the smallest priority at the front.

Task 8

Question: Use AI to implement a double-ended queue using collections.deque.

Prompt used for AI: Generate a Python DequeDS class using collections.deque. Include insert and remove operations at both ends, add docstrings, and demonstrate the operations.

Answer / Code:

```
from collections import deque

class DequeDS:
    """Double-ended queue supporting insertion and removal at both ends."""

    def __init__(self):
        self.items = deque()

    def insert_front(self, item):
        """Insert an item at the front."""
        self.items.appendleft(item)

    def insert_rear(self, item):
        """Insert an item at the rear."""
        self.items.append(item)

    def remove_front(self):
        """Remove and return the front item, or None if empty."""
        if not self.items:
            return None
        return self.items.popleft()

    def remove_rear(self):
        """Remove and return the rear item, or None if empty."""
        if not self.items:
            return None
        return self.items.pop()

    def display(self):
        """Display the deque contents."""
        print(list(self.items))

d = DequeDS()
d.insert_rear(10)
d.insert_front(5)
d.insert_rear(20)
d.display()
print("Front removed:", d.remove_front())
print("Rear removed:", d.remove_rear())
```

Explanation: collections.deque supports efficient insertion and removal from both ends.

Task 9 – Real-Time Application Challenge

Question: Choose the right data structure for each Campus Resource Management System feature, justify the choice in 2–3 sentences, and implement one selected feature.

Prompt used for AI: For this Campus Resource Management System, select the most appropriate data structure from

Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, and Deque for each feature:

- 1) Student Attendance Tracking, 2) Event Registration, 3) Library Book Borrowing,
- 4) Bus Scheduling, 5) Cafeteria Order Queue. Give a 2–3 sentence justification for each choice.

Then implement one selected feature as a working Python program with comments and docstrings.

Answer / Code:

Feature	Data Structure	Justification
Student Attendance Tracking and update	Hash Table	Student ID can be used as a key for quick lookup
Event Registration removed by an ID	Hash Table	Participant information can be searched and
Library Book Borrowing due-date priority	Priority Queue	Books/due records can be processed according to
Bus Scheduling connections	Graph	Routes and stops naturally form vertices and
Cafeteria Order Queue arrive	Queue	Students should be served in the order they

Selected feature: Cafeteria Order Queue

```
from collections import deque
```

```
class CafeteriaQueue:
    """Manage cafeteria orders using FIFO order."""

    def __init__(self):
        self.orders = deque()

    def add_order(self, student, item):
        """Add a new cafeteria order to the rear."""
        self.orders.append((student, item))

    def serve_order(self):
        """Serve the oldest order, or return None if no orders exist."""
        if not self.orders:
            return None
        return self.orders.popleft()

    def display_orders(self):
        """Display pending orders."""
        for student, item in self.orders:
            print(f"{student}: {item}")

cafeteria = CafeteriaQueue()
cafeteria.add_order("Asha", "Idli")
cafeteria.add_order("Rahul", "Dosa")
cafeteria.add_order("Meena", "Meals")

cafeteria.display_orders()
print("Serving:", cafeteria.serve_order())
```

Explanation: The selected structures are mapped to the dominant operation in each feature. The working example uses a Queue/Deque-based FIFO process for cafeteria orders.

Task 10 – Smart E-Commerce Platform

Question: Choose the most appropriate data structure for each e-commerce feature, justify the choice in 2–3 sentences, and implement one selected feature.

Prompt used for AI: For a Smart Online Shopping System, select the most appropriate data structure from Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, and Deque for:

- 1) Shopping Cart Management, 2) Order Processing, 3) Top-Selling Products Tracker,
 - 4) Product Search Engine, 5) Delivery Route Planning. Give a 2–3 sentence justification for each.
- Then implement one selected feature as a working Python program with comments and docstrings.

Answer / Code:

Feature	Data Structure	Justification
Shopping Cart Management	Linked List	Products can be added and removed dynamically
Order Processing	Queue	Orders are processed in the order they are placed
Top-Selling Products Tracker	Priority Queue	Products can be ranked by sales count
Product Search Engine	Hash Table	Product ID provides fast key-based lookup
Delivery Route Planning	Graph	Warehouses and delivery locations form connected routes

Selected feature: Product Search Engine

```
class ProductSearch:
    """Store and search products by product ID."""

    def __init__(self):
        self.products = {}

    def add_product(self, product_id, name, price):
        """Add or update a product using its ID."""
        self.products[product_id] = {
            "name": name,
            "price": price
        }

    def search_product(self, product_id):
        """Return product details for an ID, or None if not found."""
        return self.products.get(product_id)

    def delete_product(self, product_id):
        """Delete a product by ID and report whether it existed."""
        return self.products.pop(product_id, None) is not None

catalog = ProductSearch()
catalog.add_product("P101", "Laptop", 55000)
catalog.add_product("P102", "Headphones", 2500)

print(catalog.search_product("P101"))
print(catalog.delete_product("P102"))
print(catalog.search_product("P102"))
```

Explanation: The selected structures match the main operation: dynamic product storage, FIFO order processing, ranking, fast product-ID lookup, and route connectivity.

Sample Output / Verification

Run the programs in Python/VS Code and capture screenshots where required by the instructor. For Tasks 9 and 10, the source assignment specifically expects a mapping table, justification, and one working implementation.

- Task 1: Stack demonstrates push, pop, peek, and is_empty.
- Task 2: Queue demonstrates FIFO enqueue/dequeue behavior.
- Task 3: Linked list displays nodes in insertion order.
- Task 4: BST in-order traversal prints sorted values.
- Task 5: Hash table demonstrates insert, search, and delete with chaining.
- Task 6: Graph displays vertex-to-neighbor connections.
- Task 7: Priority queue serves the item with the smallest priority number first.
- Task 8: Deque demonstrates insertion/removal at both ends.
- Task 9: Campus resource features are mapped to appropriate structures and one feature is implemented.
- Task 10: E-commerce features are mapped to appropriate structures and one feature is implemented.